

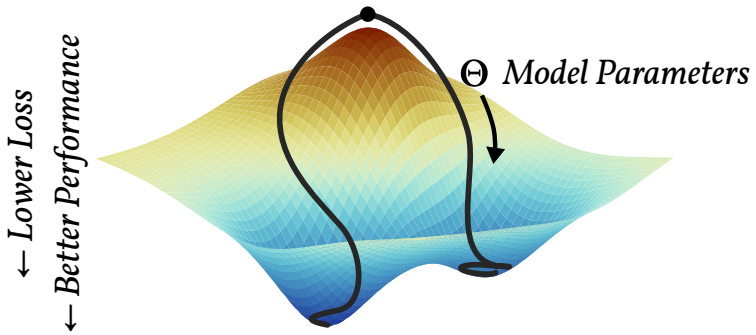
Gradient descent

Computational Neuroscience
University of Bristol

M Rule

Learning outcomes:

- ▶ Review terminology and mathematical notation for gradient descent optimization
- ▶ Be prepared to interpret certain biological learning rules as gradient-descent optimization



Gradient descent

For training pair (x, y) define

$$\hat{y} = f(x, \Theta)$$

Prediction

$$\Theta = \{\theta_1, \theta_2, \dots\}$$

Parameters

$$\mathcal{L}(\hat{y}, y)$$

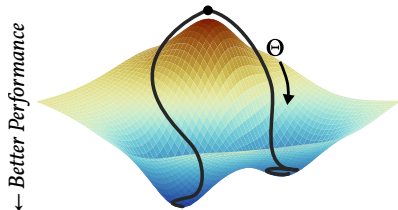
Loss (make small)

hyperparameter

Adjust Θ to **descend loss gradient**:

$$\Theta_{t+1} \leftarrow \Theta_t - \eta \cdot \nabla_{\Theta} \mathcal{L}[f(x, \Theta), y],$$

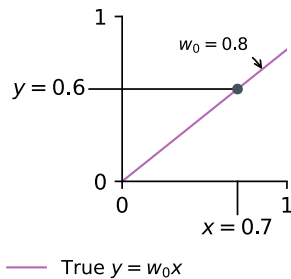
where $\eta > 0$ is the learning rate.



“empirical risk minimization”

- Minimize average loss $\langle \mathcal{L}[f(x_i, \Theta), y_i] \rangle_{i \in \text{training data}}$ over training examples

Given sample (x, y) from a linear process $y = w_0x$ with unknown w_0 ,

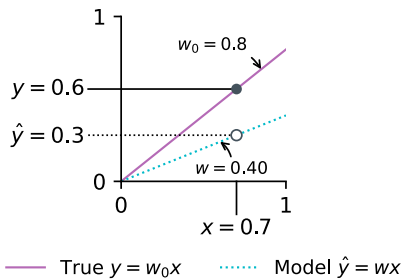


Given sample (x, y) from a linear process $y = w_0x$ with unknown w_0 ,

learn $f : x \mapsto y$

$$\hat{y}(x, w) = wx$$

prediction



Given sample (x, y) from a linear process $y = w_0x$ with unknown w_0 ,

learn $f : x \mapsto y$

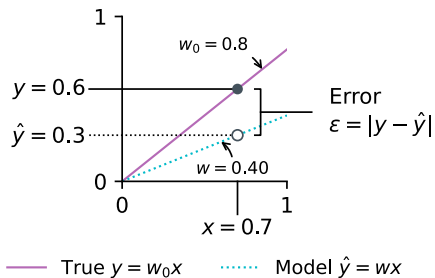
$$\hat{y}(x, w) = wx$$

prediction

Pick loss function

$$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(y - \hat{y})^2$$

loss



Given sample (x, y) from a linear process $y = w_0x$ with unknown w_0 ,

learn $f : x \mapsto y$

Pick loss function

Adjust w to decrease $\mathcal{L}$

(η : learning rate)

$$\hat{y}(x, w) = wx$$

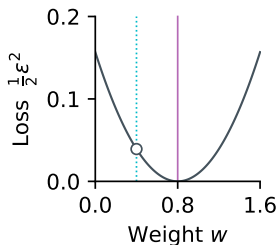
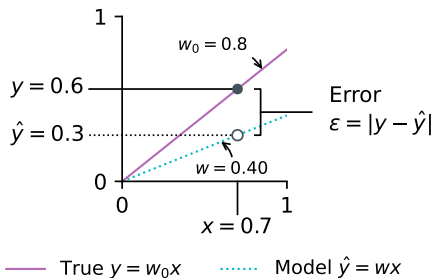
$$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(y - \hat{y})^2$$

$$w_{t+1} \leftarrow w_t - \eta \cdot \left. \frac{d\mathcal{L}}{dw} \right|_{w_t}$$

prediction

loss

gradient descent



Given sample (x, y) from a linear process $y = w_0x$ with unknown w_0 ,

learn $f : x \mapsto y$

$$\hat{y}(x, w) = wx$$

prediction

Pick loss function

$$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(y - \hat{y})^2$$

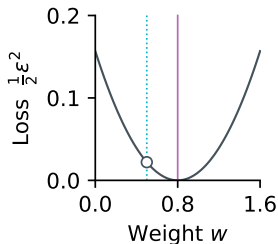
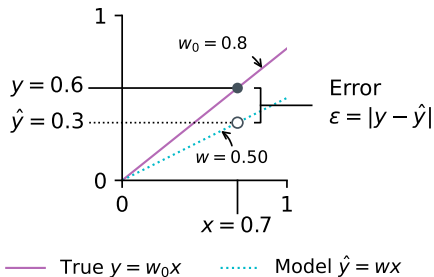
loss

Adjust w to decrease $\mathcal{L}$

(η : learning rate)

$$w_{t+1} \leftarrow w_t - \eta \cdot \left. \frac{d\mathcal{L}}{dw} \right|_{w_t}$$

gradient descent



Given sample (x, y) from a linear process $y = w_0x$ with unknown w_0 ,

learn $f : x \mapsto y$

$$\hat{y}(x, w) = wx$$

prediction

Pick loss function

$$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(y - \hat{y})^2$$

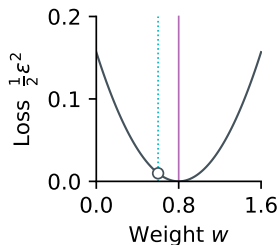
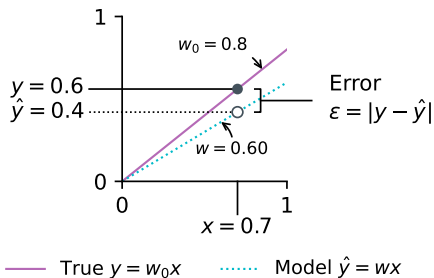
loss

Adjust w to decrease $\mathcal{L}$

(η : learning rate)

$$w_{t+1} \leftarrow w_t - \eta \cdot \left. \frac{d\mathcal{L}}{dw} \right|_{w_t}$$

gradient descent



Given sample (x, y) from a linear process $y = w_0x$ with unknown w_0 ,

learn $f : x \mapsto y$

$$\hat{y}(x, w) = wx$$

prediction

Pick loss function

$$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(y - \hat{y})^2$$

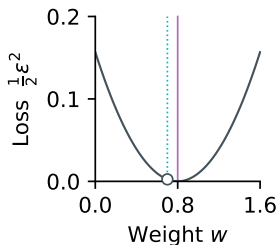
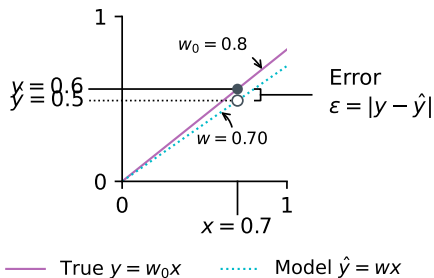
loss

Adjust w to decrease $\mathcal{L}$

(η : learning rate)

$$w_{t+1} \leftarrow w_t - \eta \cdot \left. \frac{d\mathcal{L}}{dw} \right|_{w_t}$$

gradient descent



Given sample (x, y) from a linear process $y = w_0x$ with unknown w_0 ,

learn $f : x \mapsto y$

$$\hat{y}(x, w) = wx$$

prediction

Pick loss function

$$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(y - \hat{y})^2$$

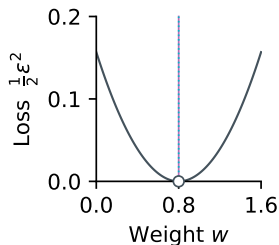
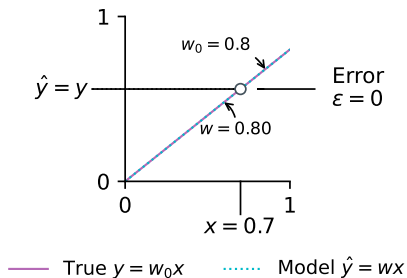
loss

Adjust w to decrease $\mathcal{L}$

(η : learning rate)

$$w_{t+1} \leftarrow w_t - \eta \cdot \left. \frac{d\mathcal{L}}{dw} \right|_{w_t}$$

gradient descent



Given sample (x, y) from a linear process $y = w_0x$ with unknown w_0 ,

learn $f : x \mapsto y$

Pick loss function

Adjust w to decrease $\mathcal{L}$

(η : learning rate)

Chain rule

$$\hat{y}(x, w) = wx$$

$$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(y - \hat{y})^2$$

$$w_{t+1} \leftarrow w_t - \eta \cdot \left. \frac{d\mathcal{L}}{dw} \right|_{w_t}$$

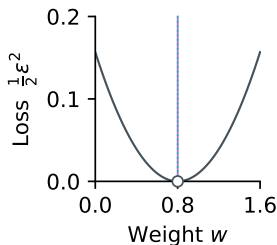
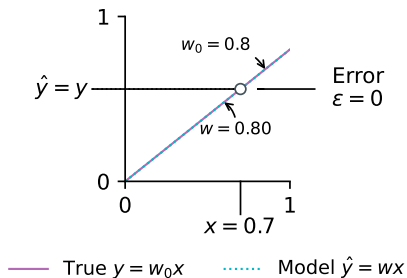
$$\frac{d\mathcal{L}}{dw} = \frac{d}{dw} [\mathcal{L} \circ \hat{y}] = \frac{d\mathcal{L}}{d\hat{y}} \frac{d\hat{y}}{dw} = -(y - \hat{y})x$$

prediction

loss

gradient descent

→ backpropagation



$$\Delta w = (\underbrace{\text{target out} - \text{predicted}}_y) \cdot \text{input}$$

work through a simple example by hand

how is this like the forward euler method?

what would this look like in code? 5/10

Select flavours of gradient descent

Say you have

- ▶ A dataset of input–output training sample pairs $\{\mathbf{x}, y^*\}_i$.
- ▶ A per-sample loss $\mathcal{L}(y_i^*, \mathbf{x}_i; \mathbf{w})$

Online gradient descent

- ▶ Update $\Delta \mathbf{w} = -\eta \cdot \partial_{\mathbf{w}} \mathcal{L}(y_i^*, \mathbf{x}_i)$ for each pair as they arrive

Batch gradient descent

- ▶ Average over multiple samples (in example below, all data), before updating
- ▶ Update $\Delta \mathbf{w} = -\eta \cdot \langle \partial_{\mathbf{w}} \mathcal{L}(y_i^*, \mathbf{x}_i) \rangle_i$

Gradient flow

- ▶ Take the $\eta \rightarrow 0$ limit to view learning as an ODE
- ▶ E.g. (batch) gradient flow $\dot{\mathbf{w}} = -\langle \partial_{\mathbf{w}} \mathcal{L}(y_i^*, \mathbf{x}_i) \rangle_i$

There may be an exam problem using gradient flow to analyze the stability of a learning rule

Gradient flow

- ▶ Take the $\eta \rightarrow 0$ limit to view learning as an ODE
- ▶ E.g. (batch) gradient flow $\dot{\mathbf{w}} = -\langle \partial_{\mathbf{w}} \mathcal{L}(y_i^*, \mathbf{x}_i) \rangle_i$

E.g. from these slides

$$\dot{w} = \langle (y - \hat{y})x \rangle = \langle (y - wx)x \rangle = \langle yx \rangle - w\langle x^2 \rangle$$

Where does it stand still (fixed point)? Set $\dot{w} = 0$

$$\langle yx \rangle = w\langle x^2 \rangle \quad \Rightarrow \quad w = \langle yx \rangle \langle x^2 \rangle^{-1}$$

(closed form least-squares solution for linear regression through the origin)

We'll work through a version with a more biological learning rule later

end

